KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

함수 2
기본값과
변수 범위

자주 쓰는 값은 미리 채워 두고
함수 안과 밖의 변수 구분하기

```
def order(menu_name, size="M"):

order("latte")

order(size="L", menu_name="tea")


count = 0

def visit(count):

    return count + 1
```

오늘의 순서

1	왜 필요한가	매번 적는 M, 사라진 total, 기본값과 안팎 그림	4–6쪽
2	기본값과 키워드 인자	여러 기본값, 칸 이름으로 넣기, 순서 규칙, print와 sorted	7–10쪽
3	지역 변수	안에서 만든 변수, 매개변수, 부를 때마다 새로	11–13쪽
4	전역 변수	밖의 변수 읽기, 같은 이름, UnboundLocalError, global과 return	14–18쪽
5	리스트와 딕셔너리 넘기기	append로 원본 바뀌기, 숫자와 비교, 재고 줄이기, copy	19–22쪽
6	실습	주문 함수, 영수증 함수, 포인트 적립	23–25쪽

3쪽은 노트북 준비, 26–27쪽은 막힐 때와 한 장 요약이에요.

들어가기

19에서 쓰던 노트북을 다시 열어요

`basics.ipynb` + Code + Markdown ▷ Run All ↻ Restart Variables .venv (Python 3.14.6)

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 basics.ipynb 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 `Shift` + `Enter` 로 실행

이번 챕터에서 다시 쓰는 것

print의 sep, end	13
`copy()`	13
`get, stock[name] -= 1`	15
`sorted(..., reverse=True)`	16
if, for	17, 18
def, return, None	19

기억이 안 나는 줄은 오른쪽 챕터를 먼저 다시 봐요.

1. 왜 필요한가

19의 함수로 쓰다 보면 두 군데서 막혀요

거의 늘 같은 값을 넣어요

[1]
```python
def order(menu_name, size):
    print(f"{menu_name} {size} 사이즈")

order("latte", "M")
order("coffee", "M")
order("tea", "M")
```
← M, M, M

```
latte M 사이즈
coffee M 사이즈
tea M 사이즈
```

[2]
```python
order("cake")
```

```
TypeError: order() missing 1 required positional argument: 'size'
```

함수 안에서 만든 값이 안 보여요

[3]
```python
def calc():
    total = 4500 * 2

calc()
print(total)
```

```
NameError: name 'total' is not defined
```

오늘은 기본값과 변수가 보이는 범위를 배워요.

1. 왜 필요한가

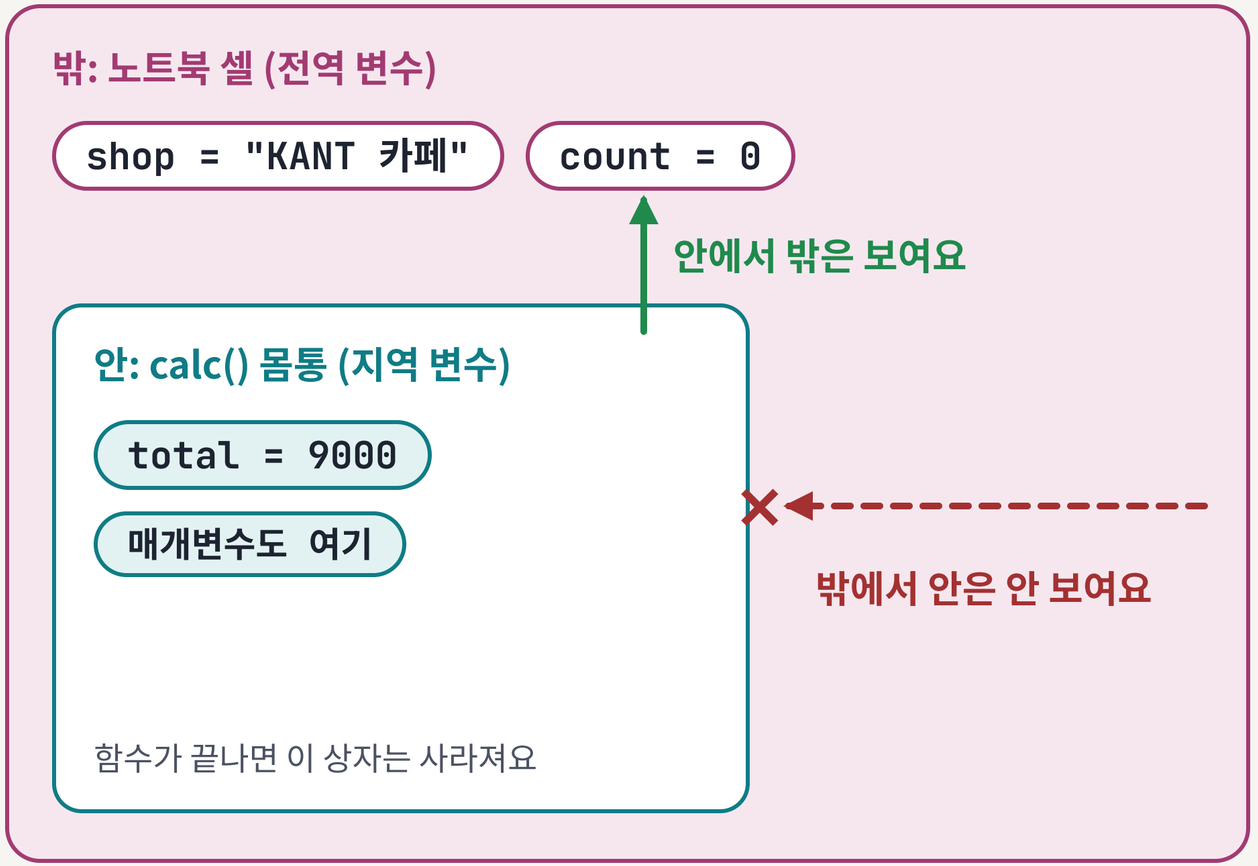
기본값은 미리 채운 칸, 변수는 안과 밖이 달라요

기본값

```
def order(menu_name, size="M"):
```

menu_name	size
꼭 넣어요	"M"

부를 때 빼면 "M"이 들어가고,
넣으면 넣은 값으로 바뀌어요.



함수 안에서는 밖이 보이고, 밖에서는 안이 보이지 않아요.

1. 왜 필요한가

def 줄에서 칸 = 값으로 기본값을 정해요

[4]

```
def order(menu_name, size="M"):                ← 기본값
    print(f"{menu_name} {size} 사이즈")


order("latte")                                 ← size를 뺐어요
order("latte", "L")                            ← size에 "L"을 넣었어요
```

Python

```
latte M 사이즈        ← 기본값 M
latte L 사이즈        ← 넣은 값 L
```

괄호 안의 =는 앞뒤를 띄우지 않고 `size="M"`처럼 붙여 써요.

기본값이 여러 개면 앞에서부터 채워요

[1]
```python
def order(menu_name, size="M", qty=1):
    print(f"{menu_name} {size} {qty}잔")

order("latte")                ← 둘 다 기본값
order("latte", "L")           ← "L"은 둘째 칸 size로
order("latte", "L", 2)        ← 2는 셋째 칸 qty로
order("latte", qty=3)         ← 칸 이름을 붙이면 qty만 바꿔요
```
Python

```
latte M 1잔
latte L 1잔
latte L 2잔
latte M 3잔     ← size는 기본값 그대로
```

넣은 값은 앞 칸부터 채우고, 넣지 않은 칸만 기본값을 써요.

2. 기본값과 키워드 인자

칸 이름을 붙이면 순서를 바꿔도 돼요

[2]
```python
order(size="L", menu_name="tea")
order("tea", qty=2, size="S")
```
← 이름 없는 값은 앞에, 이름 붙인 값은 뒤에

```
tea L 1잔
tea S 2잔
```
← 칸 이름을 보고 제자리에 들어가요

```python
order(size="L", menu_name="tea")
```

menu_name	size	qty
"tea"	"L"	1

← 넣지 않아서 기본값

2. 기본값과 키워드 인자

기본값과 키워드 인자는 뒤에 적어요

만들 때

[3]
```
def order(size="M", menu_name):
    print(menu_name, size)
```
Python

```
SyntaxError: parameter without a default follows parameter with a default
```

```
def order(menu_name, size="M"):
```
← 기본값 칸을 뒤로

부를 때

[4]
```
order(size="L", "tea")
```
Python

```
SyntaxError: positional argument follows keyword argument
```

```
order("tea", size="L")
```
← 이름 없는 값을 앞으로

두 오류 모두 실행하기 전에 SyntaxError로 알려 줘요.

키워드 인자는 이미 써 봤어요

[5]

```python
print("coffee", "latte", sep=" / ")          ← 13 · 기본값은 빈칸 하나
print("coffee", end=" ")                     ← 13 · 기본값은 줄바꿈
print("latte")
print(sorted([5000, 4500, 6500], reverse=True))  ← 16 · 기본값은 False
```

Python

```
coffee / latte
coffee latte
[6500, 5000, 4500]
```

`print()`만 쳐도 빈 줄이 찍히는 것도 end의 기본값이 줄바꿈이라서예요.

3. 지역 변수

함수 안에서 만든 변수는 밖에서 안 보여요

[1]
```python
def calc():
    total = 4500 * 2   ← 지역 변수
    print("안:", total)

calc()
print("밖:", total)
```

```
안: 9000

NameError: name 'total' is not defined
```

calc()를 부르는 동안

`total = 9000`

calc()가 끝난 뒤

~~`total = 9000`~~

끝나면 상자째 사라져요

함수 안의 값을 밖에서 쓰려면 19처럼 return으로 돌려받아요.

3. 지역 변수

매개변수도 함수 안에서만 쓰는 변수예요

[2]
```python
def greet(name):
    print(f"{name}님, 어서 오세요")

greet("kim")
print(name)
```

```
kim님, 어서 오세요

NameError: name 'name' is not defined
```

greet("kim")을 부르는 동안

name = "kim"

부를 때 넣은 값이 담긴 칸

그래서 19의 calc_total(cart, menu)에서 menu 칸은 바깥 menu와 이름만 같아요.

3. 지역 변수

부를 때마다 처음부터 새로 만들어요

[3]
```python
def add_stamp():
    stamp = 0
    stamp += 1
    print("도장", stamp)

add_stamp()
add_stamp()
add_stamp()
```

```
도장 1
도장 1
도장 1
```

← 쌓이지 않아요

첫째 부르기	둘째 부르기	셋째 부르기
stamp = 0	stamp = 0	stamp = 0
stamp += 1	stamp += 1	stamp += 1
1	1	1

값을 쌓으려면 함수 밖에 변수를 둬야 해요. 다음 파트에서 봐요.

4. 전역 변수

함수 밖에서 만든 변수는 안에서 읽을 수 있어요

[1]

```
shop = "KANT 카페"

def hello():
    print(f"{shop}에 어서 오세요")

hello()
```

Python

밖 (전역 변수)

안에 없으면 밖에서 찾아요

hello() 안

```
KANT 카페에 어서 오세요
```

가게 이름처럼 바뀌지 않는 값은 밖에 두고 안에서 읽어도 괜찮아요.

4. 전역 변수

안에서 같은 이름에 넣으면 새 지역 변수가 생겨요

[2]
```python
price = 4500

def set_price():
    price = 5000 ← 새 지역 변수
    print("안:", price)

set_price()
print("밖:", price)
```

```
안: 5000   ← 안쪽 상자
밖: 4500   ← 바깥 상자는 그대로
```

밖 price

4500

안 price

5000

이름만 같은 다른 상자예요

함수 안에서 = 로 값을 넣으면 그 이름은 지역 변수가 돼요.

4. 전역 변수

읽다가 바꾸면 UnboundLocalError가 나요

[3]

```
count = 0

def visit():
    count = count + 1
visit()
```

Python

② 오른쪽 count부터 읽어요

① = 로 넣으니 지역 변수예요

```
UnboundLocalError: cannot access local variable 'count' where it is not associated with a value
```

③ 지역 count에는 아직 값이 없어요

앞 장 규칙 그대로예요. 안에서 = 로 넣는 이름은 처음부터 지역 변수예요.

4. 전역 변수

global을 적으면 바깥 변수를 바꿔요

[4]
```python
count = 0

def visit():
    global count ← 바깥 count를 쓴다는 뜻
    count = count + 1

visit()  ← count 1
visit()  ← count 2
visit()  ← count 3
print(count)
```

```
3
```

밖 count

3

global은 있다는 것만 알아 둬요. 다음 장에서 더 나은 방법을 봐요.

4. 전역 변수

바깥 값을 바꿀 땐 return으로 돌려받아요

global로 바꾸기

[4]
```python
count = 0

def visit():
    global count
    count = count + 1

visit()
visit()   ← count가 바뀌는지 안 보여요
visit()
print(count)
```

```
3
```

return으로 돌려받기

[5]
```python
def visit(count):
    return count + 1

count = 0
count = visit(count)
count = visit(count)  ← 바뀌는 줄이 보여요
count = visit(count)
print(count)
```

```
3
```

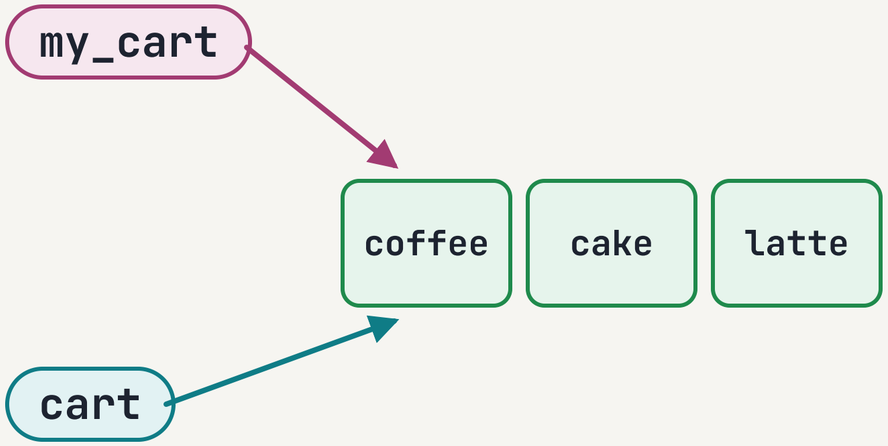
함수는 받은 값으로 계산해서 돌려주기만 해요. 이쪽을 권해요.

리스트를 넘기고 안에서 append하면 원본이 바뀌어요

[1]
```python
def add_item(cart, name):
    cart.append(name)

my_cart = ["coffee"]
add_item(my_cart, "cake")
add_item(my_cart, "latte")
print(my_cart)
```

```
['coffee', 'cake', 'latte']
```
← return 없이도 바뀌었어요



이름 두 개, 리스트 하나

my_cart와 cart는 리스트 하나를 같이 가리켜요.

5. 리스트와 딕셔너리 넘기기

숫자는 안에서 바꿔도 밖이 그대로예요

[2]
```python
def add_one(n):
    n += 1   ← n에 새 값 2를 넣어요
    print("안:", n)

x = 1
add_one(x)
print("밖:", x)
```

```
안: 2
밖: 1
```



`n += 1`	**새 값을 넣어요**	밖은 그대로
`cart.append(...)`	**받은 리스트를 고쳐요**	밖도 바뀌어요
`stock[name] -= 1`	**받은 딕셔너리 칸을 고쳐요**	밖도 바뀌어요

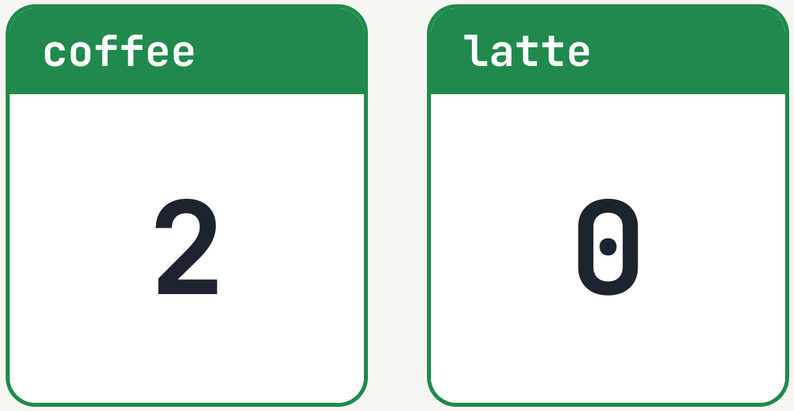
딕셔너리를 넘기면 재고를 바로 줄일 수 있어요

[3]
```python
def sell(stock, name):
    stock[name] -= 1   ← 칸만 고쳐요

stock = {"coffee": 3, "latte": 1}
sell(stock, "latte") ← latte 1 → 0
sell(stock, "coffee") ← coffee 3 → 2
print(stock)
```

```
{'coffee': 2, 'latte': 0}
```

stock



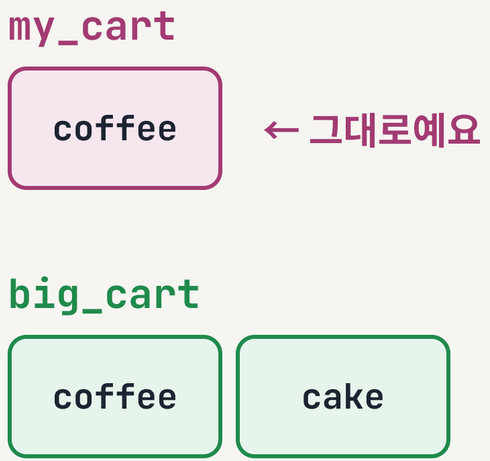
받은 딕셔너리의 칸만 고쳤으니 global이 필요 없어요.

원본을 지키려면 copy()로 새로 만들어 돌려줘요

[4]
```python
def with_item(cart, name):
    new_cart = cart.copy()  ← 복사본
    new_cart.append(name)
    return new_cart

my_cart = ["coffee"]
big_cart = with_item(my_cart, "cake")
print(my_cart)
print(big_cart)
```

```
['coffee']
['coffee', 'cake']
```



원본을 고칠지, 새로 만들어 돌려줄지 정하고 써요.

6. 실습

주문 함수: 기본값과 재고 딕셔너리

[1]

```python
def take_order(stock, name, qty=1):
    if stock[name] < qty:
        print(f"{name} {qty}개: 재고가 {stock[name]}개뿐이에요")
        return False
    stock[name] -= qty
    print(f"{name} {qty}개 나갔어요")
    return True

stock = {"coffee": 3, "latte": 1, "cake": 2}
take_order(stock, "coffee", 2)
take_order(stock, "latte")
take_order(stock, "latte")
take_order(stock, "cake", qty=3)
print(stock)
```

← 기본값: 한 개 주문은 빼고 불러요

← 모자라면 여기서 끝나요

← 받은 딕셔너리 칸을 줄여요

Python

```
coffee 2개 나갔어요
latte 1개 나갔어요
latte 1개: 재고가 0개뿐이에요
cake 3개: 재고가 2개뿐이에요
{'coffee': 1, 'latte': 0, 'cake': 2}
```

← 밖의 stock이 줄었어요

6. 실습

영수증 함수: 함수 안에서 함수 부르기

[2]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}

def calc_total(cart, menu):
    total = 0
    for name, qty in cart.items():
        total += menu[name] * qty
    return total

def print_receipt(cart, menu, rate=0):
    total = calc_total(cart, menu)   ← 위에서 만든 함수
    sale = total * rate // 100
    for name, qty in cart.items():
        print(f"{name:<7}{qty}개 {menu[name] * qty:>7,}원")
    print(f"할인   {rate:>2}% {sale:>7,}원")
    print(f"결제       {total - sale:>7,}원")
```
Python

[3]
```python
cart1 = {"coffee": 2, "cake": 1}
cart2 = {"latte": 3, "cake": 2}
print_receipt(cart1, menu)
print("---")
print_receipt(cart2, menu, rate=10)
```

```
coffee 2개    9,000원
cake   1개    6,500원
할인     0%        0원
결제          15,500원
---
latte  3개   15,000원
cake   2개   13,000원
할인    10%    2,800원   ← rate=10
결제          25,200원
```

할인이 없으면 rate를 빼고, 있을 때만 rate=10처럼 넣어요.

6. 실습

적립 함수: get과 기본값으로 포인트 쌓기

[4]
```python
def add_point(points, name, amount, rate=5):
    points[name] = points.get(name, 0) + amount * rate // 100

points = {"kim": 300}
add_point(points, "kim", 9000)
add_point(points, "lee", 6500)
add_point(points, "lee", 10000, rate=10)
print(points)
```

Python

← 평소 5%, 행사 때만 rate=10

처음 온 손님은 0에서 시작 (15)

```
{'kim': 750, 'lee': 1325}
```

부르기	더한 값	합
kim 9000	**+ 450**	750
lee 6500	**+ 325**	325
lee 10000, rate=10	**+ 1000**	1325

return 없이 받은 딕셔너리에 쌓여요

정리

막히면 오류의 마지막 줄을 봐요

보이는 것	이유	고치기
`SyntaxError: parameter without a default follows ...`	def 줄에서 기본값 칸을 앞에 뒀어요	**기본값 칸을 뒤로**
`SyntaxError: positional argument follows keyword ...`	부를 때 이름 없는 값을 뒤에 적었어요	**위치 인자를 앞으로**
`NameError: name 'total' is not defined`	함수 안의 지역 변수를 밖에서 썼어요	**return으로 돌려받기**
`UnboundLocalError: cannot access local variable`	안에서 바깥 변수를 읽다가 바꿨어요	**인자로 받고 return**
안에서 바꿨는데 밖이 그대로예요	안에서 = 로 새 지역 변수를 만들었어요	**x = 함수(x)로 받기**
함수에 넘긴 리스트가 바뀌었어요	받은 리스트를 append로 고쳤어요	**copy()로 복사해 쓰기**
부를 때마다 값이 처음으로 돌아가요	지역 변수는 부를 때마다 새로 생겨요	**값을 밖에 두고 넘기기**

헷갈리면 함수 안과 밖에서 print로 같은 변수를 찍어 비교해요.

한 장 요약

이름	모양	기억할 것
기본값	`def f(a, b="M"):`	기본값 칸은 def 줄 뒤쪽에
키워드 인자	`f(1, b=2)`	키워드 인자는 부를 때 뒤쪽에
지역 변수	`total = 9000`	함수 안에서 만들고, 끝나면 사라져요
전역 변수	`shop = "KANT"`	함수 밖에서 만들고, 안에서 읽어요
바깥 값 바꾸기	`x = f(x)`	global보다 return으로 돌려받아 바꿔요
리스트, 딕셔너리	`cart.append(x)`	넘긴 리스트는 원본이 바뀌어요
원본 지키기	`cart.copy()`	원본을 지키려면 복사해서 써요

다음 21에서는 함수를 값처럼 넘기는 lambda를 배워요.